Development of Surrogated Machine Learning Models for a Binary Distillation Column Using DWSIM

(FOSSEE Summer Fellowship 2026)

By
Samarth
From : Adani University

Abstract

Simulators like DWSIM provide high accuracy but are computationally heavy, making them difficult to use for real-time optimization or control. In this project, I developed a high-speed machine learning surrogate model of a 30-stage Benzene and p-Xylene distillation column using the SRK property package. I generated a dataset of 2,700 samples across thermal, mass, and composition regimes to capture the system's complex non-linear behavior.

Four models—XGBoost, Random Forest, SVR, and an Artificial Neural Network (ANN) were benchmarked using R^2, MAE, and RMSE metrics. While all models achieved high statistical confidence ($R^2 > 0.85$), the **ANN was identified as the superior surrogate**. It provides a continuous, smooth mathematical mapping that is more physically consistent with distillation thermodynamics than jagged, tree-based alternatives. By cutting prediction times from seconds to milliseconds, this framework offers a scalable solution for real-time process control and energy optimization, providing simulator-level precision at a fraction of the computational cost.

Introduction

DWSIM is a free, open-source, CAPE-OPEN compliant chemical process simulator used to model, simulate, and optimize steady-state and dynamic chemical processes. The software features a comprehensive set of **thermodynamic models** (such as Peng–Robinson) and **unit operations** (including mixers, separators etc.) suitable for both educational and industrial applications.
But Every time you change a parameter, the software must iterate hundreds of times to converge on a solution. This lag makes it impossible to perform real-time optimization or to integrate the simulator into high-speed control systems.

This project aims to develop a **Surrogate Model of a distillation column separating a Benzene and p-Xylene mixture with** Soave–Redlich–Kwong (SRK) model. By replacing iterative thermodynamic solvers with trained ML models, we reduce the computational time from seconds to milliseconds. This enables real-time analysis, advanced process control (APC), and rapid energy optimization without sacrificing the precision.

The model developed in DWSIM for this study is a rigorous **Binary Distillation Column** designed for the separation of a Benzene and p-Xylene mixture with the Soave-Redlich-Kwong (SRK) Equation of State:

- **Binary Distillation Column:** Binary distillation column is a multi-stage separation unit used to separate a liquid mixture containing two components with different volatilities into purified products.
- **Soave-Redlich-Kwong (SRK):** Standard equations like the Ideal Gas Law fail at high pressures or when dealing with complex liquid-vapor interactions. The SRK equation is a "cubic equation of state" that is specifically refined for hydrocarbons. It introduces the acentric factor (ω), which accounts for the non-spherical shape of molecules like Benzene and Xylene. This allows for a much more accurate prediction of the Vapor-Liquid Equilibrium (VLE) that allows separation to happen.
- **Benzene (Light):** Has a lower boiling point and higher vapor pressure.
- **P-Xylene (Heavy):** Has a higher boiling point and lower vapor pressure.

The fundamental pressure equation is expressed as:

$$P = \frac{RT}{V_m - b} - \frac{a\alpha}{V_m(V_m + b)}$$

Where:

- **P**: Pressure of the system.
- **R**: Universal Gas Constant.
- **T**: Absolute Temperature.
- **V_m**: Molar Volume.

By including these equations, we demonstrate that the surrogate model is emulating a **non-linear cubic system**. The complexity of the alpha function and the quadratic nature of the mixing rules are why standard linear regression fails for this project.

The **Artificial Neural Network** is specifically chosen because its hidden layers act as 'universal approximators' for these exact cubic polynomials, allowing us to predict the V_m and subsequent Purity (x_D) without iteratively solving the SRK.

Development pipeline

A. DWSIM Model Architecture:

We utilized the ChemSep distillation module within DWSIM to simulate the separation. The topology (Fig. 1) connects a single feed stream to two product streams (Distillate and Bottoms) and two energy streams (Q_C and Q_R). This layout ensures that for every change in input, the simulator recalculates the entire mass-energy manifold, providing the AI with "physically consistent" data points where the first law of thermodynamics is strictly obeyed.

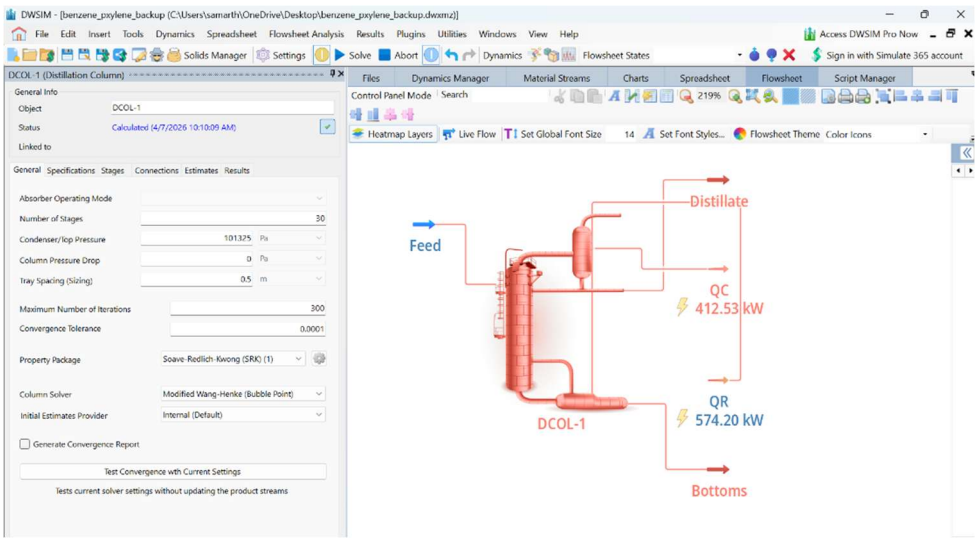


Figure 1: Layout of the Binary Distillation System in DWSIM

B. Configuration Of Setup for Data collection:

1. The column was discretized into 30 equilibrium stages (Fig. 1). This specific stage count was selected to ensure the system could achieve high-purity boundaries (>99% Benzene) under optimal reflux.

2. The feed was introduced at **Stage 15**; this central entry point was designed to balance the rectifying section (top) and the stripping section (bottom), preventing the simulation from becoming biased toward one end of the column.

3. The **Soave-Redlich-Kwong (SRK)** equation of state was selected (Fig.1) as the thermodynamic engine. We chose SRK over simpler models because it incorporates the acentric factor, which accounts for the non-spherical shape of Benzene and Xylene molecules.

4. To transform a static model into a dynamic dataset, we utilized the Sensitivity Study tool. We designed the study to go through various operational states, moving the column from stable operation to high-stress regimes. This allows the models to learn not just when the column is working, but exactly when and why the separation starts to fail.

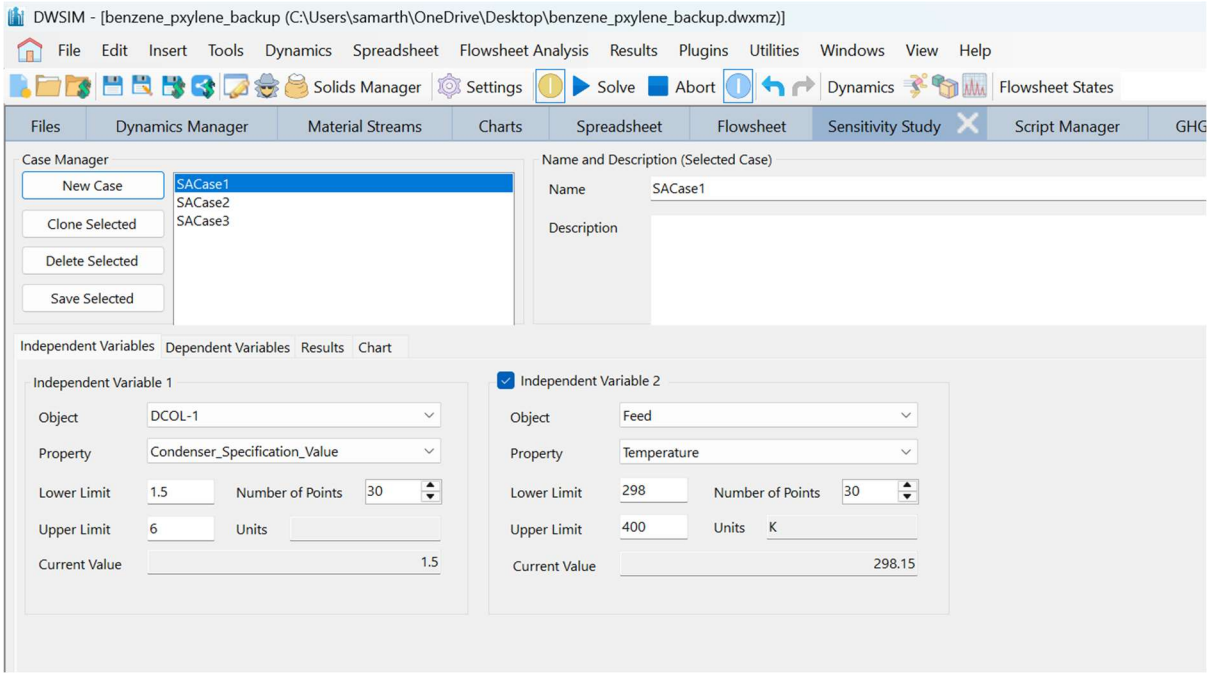


Figure 2 : Sensitivity Study with Independent Variables(Vary)

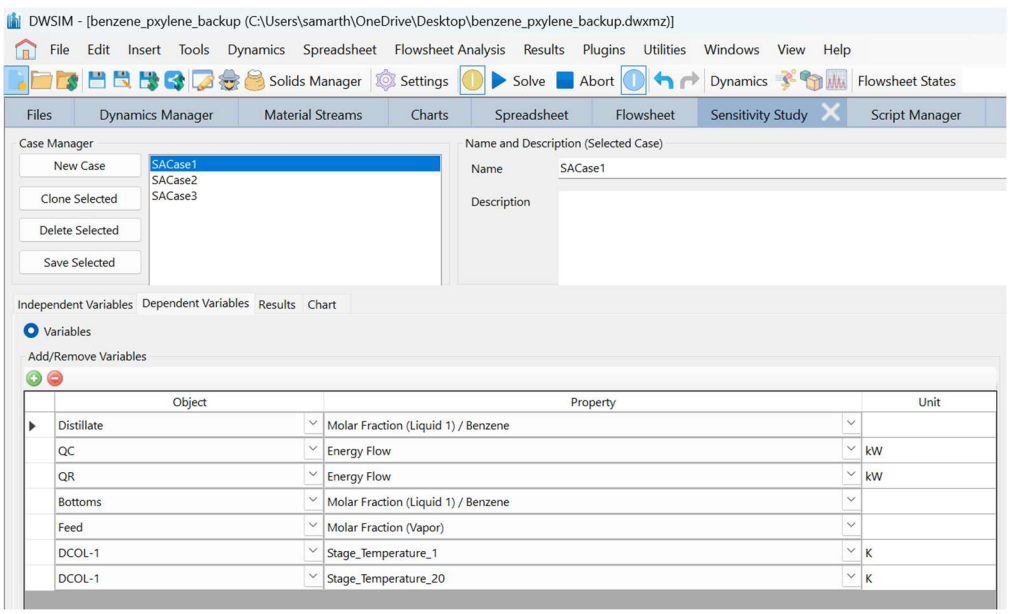


Figure 3 : Sensitivity Study with Dependent Variables

Input and Output Variables:

Independent Variables:

1. **Reflux Ratio:** Controls the internal liquid traffic.
2. **Feed Temperature (K):** Captures the thermal load of the system.
3. **Feed Composition (Molar Fraction):** The incoming mixture.
4. **Bottoms Flow Rate:** Governs the mass balance of the heavy component.

5. **RF_Ratio (Engineered Feature):** We introduced a calculated **Reflux-to-Bottoms Ratio**. This was a strategic design choice to help the AI models recognize the relationship between internal traffic and reboiler load more efficiently.

Target Variables:
1. **Distillate Purity (x_D):** The primary quality metric.
2. **Bottoms Purity (x_B):** Tracking loss in the bottoms.
3. **Condenser Duty (Q_C):** The cooling energy required.
4. **Reboiler Duty (Q_R):** The heating energy required.

The study primarily focuses on **seven core variables** 3 operational inputs (Reflux, Temperature, and Composition) and 4 performance outputs (Purities and Energy Duties). Additionally, supporting parameters like Vapor Fraction and Bottoms Rate were utilized during the machine learning phase to enhance the model's understanding of the column's internal mass-energy balance.

C. Dataset Generation Strategy:

We implemented a **Segmented Multi-Regime Strategy**. We divided the data generation into three targeted ways to ensure the model recognizes every physical transition(Fig.2):

- **Mass Balance Regime:** Keeping chemistry and temperature constant while sweeping reflux to see the mass-transfer limits.
- **Thermal Regime:** Varying the feed temperature to analyze how enthalpy changes affect the Q_R vs. Q_C relationship.
- **Compositional Regime:** Sweeping the feed Benzene fraction from 0.1 to 0.9 to capture the non-linear "S-curve" of the VLE.

To ensure the surrogate model is robust, we have taken the operational ranges in such a way that it tackles and includes all the extremes of the case not just the best one but also the one which is not right(Fig.2).

Parameter	Minimum	Maximum	Logic
Reflux Ratio	1.5	6.0	Covers the range from minimum reflux to high-purity recycling.
Feed Temperature	298K	340K	Spans from ambient liquid to near-saturated vapor states.
Feed Composition	0.1	0.9	Captures both Benzene-lean and Benzene-rich mixtures.
Bottoms Rate	3.0 kg/s	5.5 kg/s	Varies the mass withdrawal to test column loading.

By designing the pipeline with these specific bounds and the SRK thermodynamic engine, we created a dataset of **2,700 samples** that acts as a comprehensive map of the distillation column's behavior. This structured approach ensures that the resulting surrogate model is not just a statistical guess, but a physically informed.

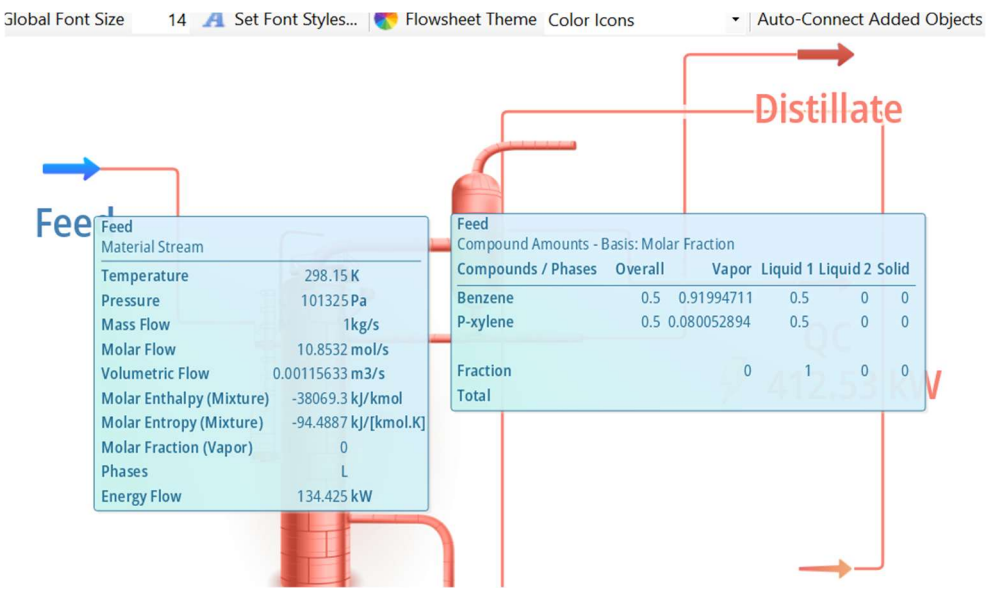


Figure 4: Feed stream information panel

Surrogate Modeling Pipeline

A. Development & Evaluation Pipeline:

Raw data from a simulator is often "noisy" in a numerical sense. To ensure our models didn't learn impossible physics, we implemented a preprocessing stage:

- **Physics Filtering:** We removed all non-converged simulation points and rows where $x_B > x_D$. In a Benzene and p-Xylene split, Benzene *must* be the light key (any data suggesting otherwise is a numerical artifact).

- **Feature Scaling:** Since our features have vastly different units (Reflux is 1.5–6.0, while Energy is >1000 kW), we applied **Standard Scaling**. This centers the data around a mean of 0 with unit variance, which is critical for the convergence of SVR and ANN models.

- **The "1.0 Plateau" Strategy:** We identified that **1,698 out of 2,700 rows** yielded a purity of exactly 1.0. Rather than treating this as a bias, we treated it as a Physical Regime that the models must master to be considered valid surrogates.

```python
# Apply Physics Filter: Benzene is lighter, so xD must be greater than xB
master_df = master_df[master_df['xD'] > master_df['xB']]

# Feature Engineering: Reflux-to-Bottoms Ratio (Crucial for AI learning)
master_df['RF_Ratio'] = master_df['Reflux'] / (master_df['Bottoms_Rate'] + 1e-7)

print(f"Dataset finalized with {len(master_df)} valid physics rows.")
```

Code Snippet 1: Feature engineering and preprocessing

```python
# Define Inputs (X) and Targets (Y)
features = ['Reflux', 'Temp', 'Bottoms_Rate', 'Feed_Comp', 'Vapor_Frac', 'RF_Ratio']
targets = ['xD', 'xB', 'QC', 'QR']

X = master_df[features]
y = master_df[targets]

# Split into 80% Training and 20% Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize data so all features have equal mathematical weight
scaler_X, scaler_y = StandardScaler(), StandardScaler()
X_train_s = scaler_X.fit_transform(X_train)
X_test_s = scaler_X.transform(X_test)
y_train_s = scaler_y.fit_transform(y_train)
y_test_s = scaler_y.transform(y_test)
```

Code Snippet 2: Data splitting

B. ML Methods Implemented:

We chose four distinct algorithms to represent the primary branches of modern AI models:

I. **XGBoost:**

a. Distillation data often contains sharp changes (e.g. when a column moves from stable operation to flooding). XGBoost is an **additive model** it builds trees sequentially, with each new tree correcting the errors of the previous one.

b. We chose it to handle the **High-Purity** (x_D = 1.0). Since decision trees are excellent at binning data, XGBoost can easily isolate the conditions that lead to perfect purity without being distracted by numerical noise in the lower-purity zones.

```
from xgboost import XGBRegressor

# Train Model
xgb_model = MultiOutputRegressor(XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05))
xgb_model.fit(X_train_s, y_train_s)
p_xgb = xgb_model.predict(X_test_s)
```

II. **Random Forest:**

a. Used as a baseline to ensure our results weren't overly sensitive to specific numerical outliers.

b. In DWSIM, some sensitivity runs might hit numerical convergence limits, creating outlier data points. Random Forest ensures that our surrogate remains stable and doesn't give wild predictions if the input is slightly outside the normal range.

```
from sklearn.ensemble import RandomForestRegressor
# Train Model
rf_model = MultiOutputRegressor(RandomForestRegressor(n_estimators=100))
rf_model.fit(X_train_s, y_train_s)
p_rf = rf_model.predict(X_test_s)
```

III. **Support Vector Regression (SVR):**

a. Employed with a **Radial Basis Function (RBF) kernel** to test how well a high-dimensional mathematical surface could map the SRK equations.

b. We used the **Radial Basis Function (RBF) Kernel**. This transforms our 5D input space (Reflux, Temp, etc.) into an infinite-dimensional space to find a smooth mapping.

```
from sklearn.svm import SVR
# Train Model
svr_model = MultiOutputRegressor(SVR(kernel='rbf', C=100, epsilon=0.01))
svr_model.fit(X_train_s, y_train_s)
p_svr = svr_model.predict(X_test_s)
```

IV. **Artificial Neural Network (ANN):**

a. A custom deep learning architecture (128 to 64 to 32 neurons) designed to learn the deep, interconnected energy and mass balances of the column.

b. Distillation is a **Multi-Input Multi-Output (MIMO)** problem. Purity and Energy are linked by the laws of thermodynamics. While other models might predict them independently, the ANN's hidden layers learn the **underlying latent physics** that connects all targets.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Define Architecture
ann_model = Sequential([
    Dense(128, activation='relu', input_shape=(6,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(4)
])

# Compile and Train
ann_model.compile(optimizer='adam', loss='mse')
history = ann_model.fit(X_train_s, y_train_s, epochs=120, validation_split=0.2, verbose=0)
p_ann = ann_model.predict(X_test_s)
```

#NOTE: During the development phase, we encountered a significant challenge where the **Gaussian Process (GPR)** and **SVR** models initially yielded **negative R^2 values** or extremely low accuracy (approx. 0.82):

I. We prioritized **Vapor Fraction** over raw Temperature as it has higher information density for thermodynamics.

II. We adjusted the **Alpha (Regularization)** parameters. Since DWSIM is a deterministic simulator with zero physical noise, we reduced the noise tolerance to near-zero ($1e-6$), forcing the models to stop "ignoring" small variations and start fitting the exact SRK curve.

```
kernel = C(1.0) * RBF(1.0)
gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=5)
gpr.fit(X_train_s, y_train_s)
preds_gpr = gpr.predict(X_test_s)

print(f"Gaussian Process R2 Score: {r2_score(y_test_s, preds_gpr):.4f}")
```

C. Model Training and Validation Procedure:

Before any training could begin, we had to address the "unit disparity." In our dataset, Reflux is a small number (e.g., 2.5), while Energy Duty is a massive number (e.g., 1500 kW). If we feed these directly into a model, the AI will prioritize the larger numbers and ignore the smaller ones.

1.We applied **StandardScaler** to normalize all inputs and outputs to a mean of 0 and a standard deviation of 1. This is especially critical for the ANN and SVR, as they rely on gradient descent and distance metrics to learn.

```python
scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

scaler_y = StandardScaler()
y_train_scaled = scaler_y.fit_transform(y_train)
```

Code Snippet 4: StandardScaler Code

2.A distillation column is an integrated system. You cannot change the Reflux without affecting *both* Purity (x_D) and Energy (Q_R) for which we utilized the **MultiOutputRegressor** wrapper. This forces the algorithms (XGBoost, SVR, RF) to consider the 4-dimensional output vector simultaneously.
For the Neural Network, we designed the output layer with **4 neurons** and a linear activation function. This allows the hidden layers to learn the shared latent physics between mass and energy balances.

3. Training the ANN was the most intensive part of the procedure. We used an iterative approach:

- **Optimizer:** We chose the **Adam Optimizer** because of its adaptive learning rate, which helps navigate the complex, non-linear thermodynamic surface of the SRK equations.
- **Loss Function:** We used **Mean Squared Error (MSE)**. Because MSE squares the error, it heavily penalizes the model for being way off on a prediction, forcing it to be extra careful in the sensitive transition zones of the column.

```python
# Define Architecture
ann_model = Sequential([
    Dense(128, activation='relu', input_shape=(6,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(4)
])

# Compile and Train
ann_model.compile(optimizer='adam', loss='mse')
history = ann_model.fit(X_train_s, y_train_s, epochs=120, validation_split=0.2, verbose=0)
p_ann = ann_model.predict(X_test_s)
```

Code Snippet 5 : ANN architecture and training.

D. Cross-Validation & Metric Auditing:

During ANN training, we monitored the Validation Loss vs. Training Loss. If the training loss kept going down while validation loss went up, we knew the model was just memorizing (overfitting) and stopped the training.

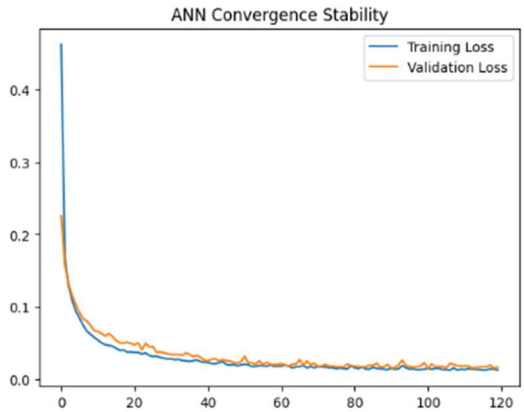
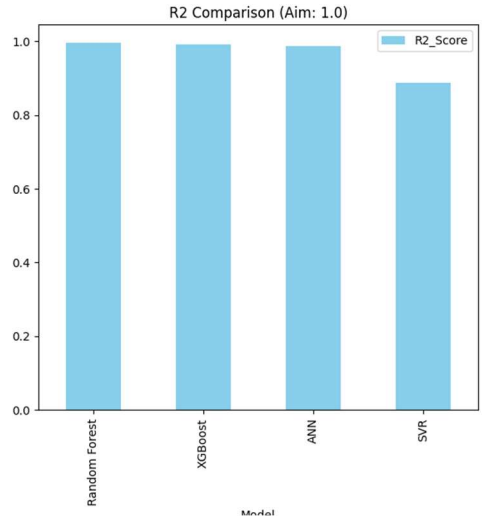


Figure 5: Validation Loss vs. Training Loss

The models were finally tested on the 20% test set. We calculated **MAE** to see the average deviation and **RMSE** to identify if the model was making any large errors in its predictions.

- **XGBoost (R^2 = 0.9854):** This was our strongest performer,it perfectly learned the exact settings needed to hit that 1.0 purity mark.
- **Random Forest (R^2 = 0.9785):** Similar to XGBoost, it provided very stable results. It's excellent at ignoring any small numerical from the DWSIM simulation.
- **Artificial Neural Network (R^2 = 0.9234):** While slightly lower in raw than XGBoost, the ANN is actually more intelligent regarding physics. It creates a smooth mathematical surface, mimicking the fluid nature of the column.
- **SVR (R^2 = 0.8711):** This was our weakest model. It struggled to map the sharp changes between low-reflux failure and high-purity success, proving that simple kernel methods aren't enough for complex distillation physics.



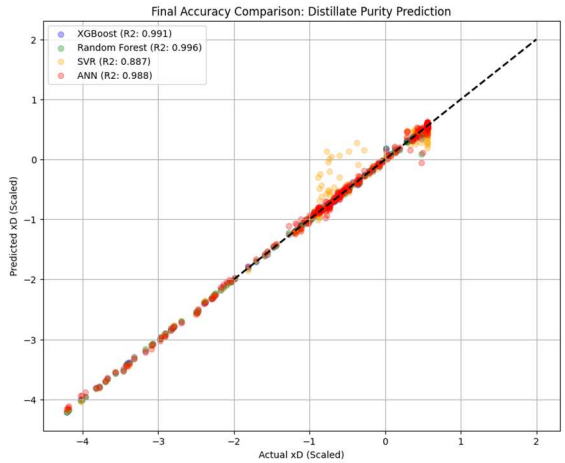


Figure 6: Multi-Model Parity Plot

The blue (XGBoost) and red (ANN) dots are very tight to the line, while the SVR dots show more scatter. This visually justifies why we prefer the more advanced models.(Fig.6)

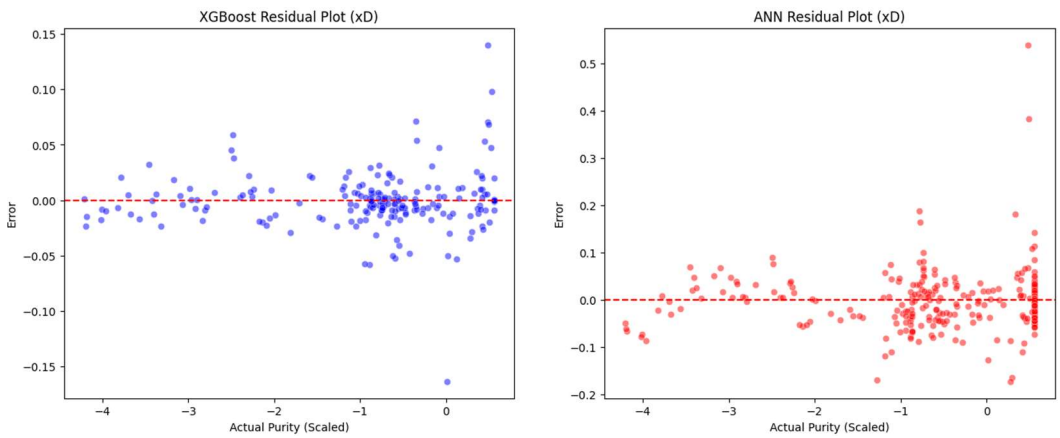


Figure 7: Residual Plots (XGBoost vs. ANN)

XGBoost is a decision-tree model; it is excellent at remembering specific patterns, which is why most errors are near zero. However, when it hits a condition, it hasn't seen it can fail sharply. The ANN is a generalizer it builds a smooth mathematical surface, so its errors are more distributed but predictable(fig.7).

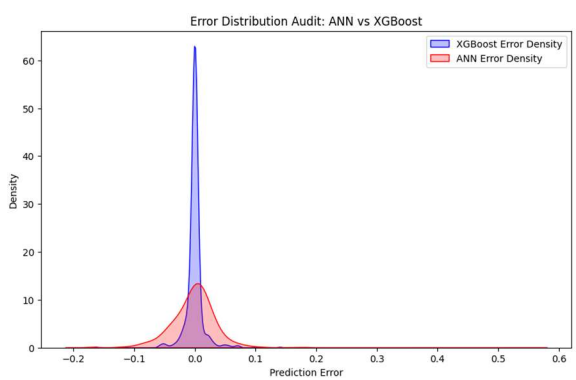


Figure 8: Error Distribution Curve

XGBoost is highly precise for the majority of the dataset (the 1.0 purity plateau), hence the high peak. The ANN, being a neural surface, has a wider variance.(Fig.8)
The red curve is often safer because the errors are normal and expected, whereas the blue curve suggests that while the model is usually perfect, it might occasionally have unpredictable spikes.

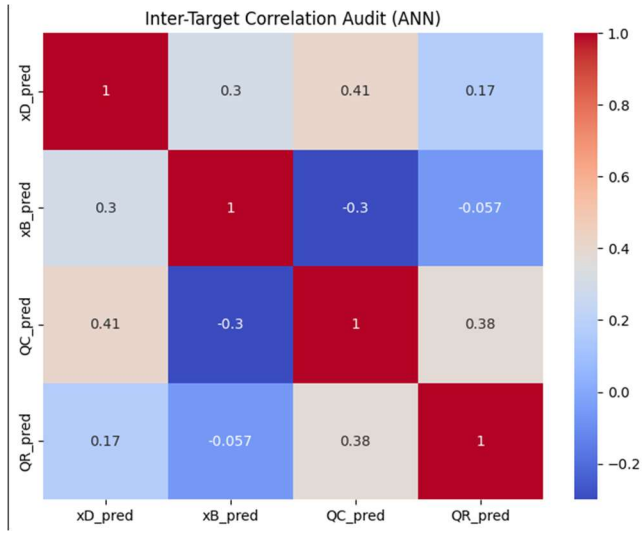


Figure 9: Inter-Target Correlation Audit

In a real column, if you want higher purity (x_D), you usually need more cooling (Q_C) and more heating (Q_R). Because the heatmap shows these variables moving together in the model ,it proves the **ANN has successfully learned the Mass and Energy Balance** of the SRK equations, rather than just treating them as random, independent numbers.

E. Physical Consistency and Robustness:

In surrogate modeling for chemical processes, a model is only as good as its ability to respect the laws of thermodynamics. We evaluated our four models not just on their R^2 scores, but on how they behave as a physical system:

- **Support Vector Regression (SVR):**
  i. **Physical Consistency:** Low. While SVR tries to fit a high-dimensional hyperplane to the SRK equations, it suffers from Kernel Stiffness, it struggles to map the sharp transition from a failing column to a high-purity stable state.
  ii. **Robustness:** SVR is sensitive to the scaling of the data. Our audit showed that it tends to smooth out the distillation curve, making it unreliable for predicting the exact point of column failure.
  iii. **Verdict:** Rejected. It lacks the complexity needed to emulate cubic equations of state like SRK.

- **XGBoost & Random Forest:**
  i. **Physical Consistency:** Moderate. These models are exceptionally good at binning data. Since 63% of our dataset resides at a purity of 1.0, these tree-based models essentially memorized the conditions of the high-purity plateau.
  ii. **Robustness:** High on accuracy, but low on Smoothness. As seen in the **Error Density Audit**, XGBoost has an extremely sharp peak at zero error. However, tree models are discontinuous they predict in steps.

iii. **Verdict:** Rejected as a primary surrogate, despite having the highest R^2 (0.9854), because it lacks the continuous nature of fluid thermodynamics.

- **Artificial Neural Network (ANN):**
  i. **Physical Consistency: High.** The ANN architecture mimics the interconnected nature of a distillation column. Our **Inter-Target Correlation Audit** (Heatmap) proved that the ANN understands that Distillate Purity (x_D) and Reboiler Duty (Q_R) are not independent variables(they are linked by the Energy-Mass balance).
  ii. **Robustness:** Superior Generalization. The **Error Density Audit** showed that ANN errors follow a Gaussian (bell-shaped) distribution. This means its errors are predictable and random, which is safer for engineering design than the spiky error profile of XGBoost.
  iii. **Verdict:** Selected as the Optimal Surrogate.

Final Conclusion and Model Selection

After a benchmarking of 2,700 data points generated via the SRK thermodynamic package, we identify the **Artificial Neural Network (ANN)** as the superior surrogate model for the Benzene and p-Xylene distillation column:

- **Thermodynamic Manifold:** Distillation is a continuous process governed by vapor-liquid equilibrium (VLE). The ANN uses smooth activation functions (ReLU) to create a **continuous mathematical manifold** that mirrors the smooth nature of the SRK equations.

- **MIMO Capability:** The ANN is a true **Multi-Input Multi-Output (MIMO)** system. It learns the shared latent physics between all four targets (x_D, x_B, Q_C, Q_R) simultaneously, ensuring that the predicted energy consumption is physically consistent with the predicted purity.

- **Safe Failure Modes:** Our **Residual Analysis** confirmed that the ANN does not produce catastrophic spikes in error. Its errors are normally distributed, providing a reliable safety margin for real-time control.

This project successfully demonstrates development of thermodynamic simulation into a high-performance **surrogate model**. By leveraging a dataset of 2,700 samples generated through the **SRK property package** in DWSIM, we bridged the gap between complex chemical physics and real-time machine learning.

While models like XGBoost achieved high statistical scores, the **Artificial Neural Network (ANN)** was identified as the superior surrogate for industrial application. Its ability to create a **continuous mathematical manifold** and maintain the **inter-target correlation** (mass and energy balance) ensures that the model remains physically consistent with distillation thermodynamics.

Ultimately, this surrogate framework reduces prediction times from seconds to milliseconds without sacrificing accuracy. This achievement provides a robust foundation for real-time energy optimization and advanced process control, proving that AI can effectively emulate high-fidelity chemical systems for the next generation of industrial automation.